

4WEB3 – Développement WEB 3

Repartitor*Projet***Consignes**

- ▶ Lisez ce document dans son intégralité avant de commencer à écrire une seule commande ou ligne de code.
- ▶ Posez vos questions à votre enseignant de labo : c'est lui votre « client », il pourra clarifier ses attentes.

1 Présentation et objectifs

Ce projet vous demande d'implémenter un outil de gestion des textes pour un service de traduction.

1.1 Cas d'utilisation

Une traduction nécessite deux étapes opérée par des personnes différentes : la traduction elle-même, puis le *traitement de texte*, ici désigné sous le terme d'édition¹, pour la mise en page finale.

Le rôle du répartiteur est d'attribuer à chaque traducteur et éditeur les textes à traiter afin de satisfaire les contraintes de temps fournies par le client. Ce projet doit permettre au répartiteur d'avoir une vision globale des textes et de pouvoir les répartir à chaque intervenant.

1.2 Technologies

Les technologies que vous **devez** utiliser sont celles qui ont été vues en labo. À savoir :

- ▶ VueJS
- ▶ Vue-Router
- ▶ Pinia
- ▶ Supabase

Nous ne souhaitons pas brider votre créativité et votre envie de découvrir. N'hésitez pas à en utiliser d'autres, en particulier pour le CSS (cf 9.2 page 10).

2 Initialisation du code

Créez un nouveau projet

```
>_ npm create vue@latest
```

Nommez le **repartitor-g12345** où 12345 est remplacé par votre numéro d'étudiant.

1. L'édition recouvre en réalité plusieurs aspects : la mise en page que nous considérons ici en fin de processus, mais aussi la correction de la langue originale à son début, étape ignorée pour ce projet.

Sélectionnez les éléments suivants :

- Router
- Pinia
- Linter

Header et menu

Ajoutez sur votre site :

- un `header` avec le titre de votre projet,
- un `footer` avec votre nom et matricule,
- 4 routes pour trois composants *views* placés dans un dossier `src/views`. Ces composants correspondent respectivement à une vue *Accueil*, une vue *Gestion des rôles*, une vue *Gestion des textes* et une vue *Statistiques*. Leur contenu n'est pas important à ce stade, vous pouvez ajouter un titre avec un contenu quelconque. Un menu contenant un lien pour chaque page permet de naviguer entre les composants précédents.
- Ajoutez un bouton "Connexion avec Google" (qui ne fait rien pour le moment).



FIGURE 1 – Proposition d'entête (`header`).

3 Initialisation de la base de données

Comment fait en laboratoire, commencez par créer un nouveau projet dans Supabase.

L'archive `Database.zip` disponible sur poÉSI contient les requêtes SQL qui vont créer vos tables, mettre en place les règles de politique d'accès (« policy ») ainsi que les fichiers `csv` qui vont nourrir vos tables. Si le schéma vous semble complexe, vous pourrez consulter le *schema visualizer* intégré à Supabase dès que vos tables seront configurées.

repartitor-tables.sql Contient les commandes de créations des tables de la base, ainsi que la définition de la politique d'accès à celles-ci. En l'occurrence, elles donnent accès à toutes les opérations CRUD sur les tables précédemment décrites à un utilisateur connecté.

repartitor-views.sql Contient deux exemples de création de vues et de fonctions utilisant les tables. En effet, Supabase utilise une API PostgREST qui limite ce qu'il est possible de faire comme requêtes (par exemple pas de clause `GROUP BY`, pas d'auto-jointure...). Définir en SQL (ici PostgreSQL) des vues permet de contourner ce problème.

3.1 Description des tables

text La table qui contient les métadonnées sur un texte lorsqu'il est fourni par le client au service de traduction : identifiant (`id`), cote (`cote`), titre (`title`), date de réception (`received`), date limite (`deadline`), nombre de mots (`wordcount`), le cas échéant, le texte auquel il fait suite (`precedent`).

translator La table qui contient les informations sur les traducteurs : trigramme (`id`), nom et prénom (`lastname` et `firstname`), ainsi que le nombre de mot qu'il est attendu qu'il traduise par jour (`expectedthroughput`).

translation La table d'association qui contient les informations sur les attributions de textes aux traducteurs : le texte et le traducteur concernés (**text** et **translator**), la date d'attribution (**attributed**), la date limite de traduction (**deadline**), et le cas échéant la date où la traduction a été terminée (**finished**).

editor La table qui contient les informations sur les éditeurs : trigramme (**id**), nom et prénom (**lastname** et **firstname**).

edition La table d'association qui contient les informations sur les attributions de textes aux éditeurs : le texte et éditeur concernés (**text** et **editor**), la date d'attribution (**attributed**), s'il est connu le trigramme du traducteur chargé de la traduction (**translated_by**), et le cas échéant la date où l'édition a été terminée (**finished**).

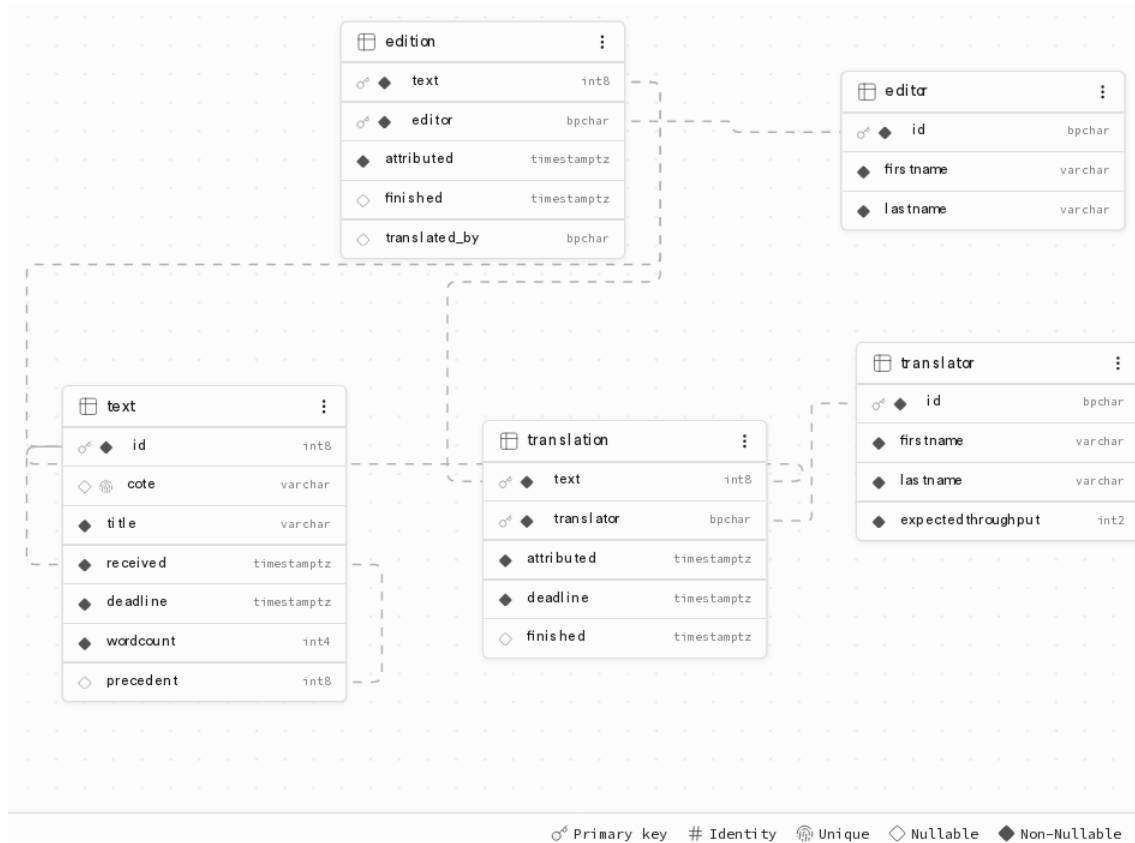


FIGURE 2 – Vue du schéma de la base de données (Database / Schema Visualizer).

3.2 Données

Toutes les données sont présentes dans le dossier **data/**. Utilisez l'import **csv** de **Supabase** pour associer les données aux tables correspondantes (voir figures 4 page suivante).

En raison des contraintes de clef étrangère, il faut faire attention à l'ordre d'import des données. Par exemple, les liens d'édition doivent être importés les derniers.

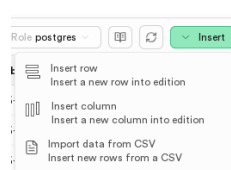


FIGURE 3 – Menu d'import CSV.

Add data to edition

Upload CSV

Paste text

Upload a CSV or TSV file. The first row should be the headers of the table, and your headers should not include any special characters other than hyphens (-) or underscores (_).

Tip: Datetime columns should be formatted as YYYY-MM-DD HH:mm:ss

editions.csv

Remove File

Configure import data

Preview data to be imported

A total of 72 rows will be added to the table "edition"

Here is a preview of the data that will be added (up to the first 20 columns and first 20 rows).

text	editor	attributed	finished	translated_by
1	HRO	2026-04-12 06:18:51		CTO
2	NGR	2026-04-13 03:55:00		GNO
3	JMC	2026-03-29 11:12:38	2026-04-05 00:25:36	ANE
4	MZA	2026-04-03 02:42:52	2026-04-11 20:39:34	GRU
5	HRO	2026-03-29 22:15:22	2026-04-06 11:42:20	TLA
6	CKL	2026-04-04 10:44:50	2026-04-09 12:15:13	ALA
7	YBA	2026-03-30 21:02:06		HHE

Cancel

Import data Ctrl+↵

FIGURE 4 – Import CSV (Table editor / Insert / Import data from CSV).

4WEB3 – Repartitor – page 4/11

4 Authentification

Lors des laboratoires, vous avez utilisé *OAuth* pour vous connecter à Supabase en utilisant *git.esi-bru.be* comme *provider*. Nous vous demandons la même chose dans ce projet à la différence que vous allez utiliser Google comme *provider*.

La configuration Google n'est pas compliquée, mais il faut trouver les bons liens.

4.1 Création d'un projet

<https://console.cloud.google.com/projectcreate>. Créez un nouveau projet. Attention, il est important de sélectionner la HE2B comme organisation.

4.2 Création et configuration d'un identifiant OAuth

1. Dans la section "Accès rapide", au sein du menu "API et service", cliquez sur "Identifiants".
2. Utilisez le bouton "+ Créer des identifiants" en choisissant le type ID Client OAuth.
3. Il vous demandera peut-être de créer un écran de consentement du projet :
 - ▶ Choisissez comme nom d'application *Repartitor-g12345*.
 - ▶ Utilisez votre adresse *@etu.he2b.be* comme contact et développeur.
 - ▶ Précisez une audience "Interne".
4. Le type d'application est "Application Web".
5. Appelez ce client "Client Repartitor".
6. Ne sélectionnez pas une origine JavaScript autorisée.
7. Vous obtiendrez l'URI de redirection autorisée sur votre projet Supabase (Dashboard / Configuration / Sign In Up / sélectionnez Auth Providers Google.). Attention, **gardez cette page ouverte**.
8. Félicitations, votre *provider* est configuré. Récupérez le Client IDs et le Client Secret et insérez le dans la dernière page Supabase gardée ouverte.

Utilisez un fichier *.env* pour configurer les deux variables liées à l'URL et la PUBLISHABLE KEY. Si vous ne les avez pas gardé lors de la première configuration, vous pourrez les retrouver dans le dashboard de Supabase, sous le menu *Project settings* :

- ▶ Dans *General* pour avoir l'ID de votre projet ; l'url est alors
`VITE_SUPABASE_URL="https://<id du projet>.supabase.co"`
- ▶ Dans *API Keys* pour avoir la clef publique. Votre variable ressemblera alors à
`VITE_SUPABASE_PUBLISHABLE_KEY="sb_publishable_<suite de la clef ici>"`

4.3 Bouton de connexion et gestion de l'état de connexion

La configuration OAuth est terminée, il ne reste plus qu'à implémenter dans l'entête un bouton de connexion avec la librairie Supabase comme vous l'avez fait en laboratoire. Une fois connecté, l'identité de l'utilisateur est affiché aux côté du bouton de déconnexion.

Par ailleurs, les pages autres que l'accueil n'étant plus accessibles, il faut faire en sorte que :

- ▶ Les liens n'apparaissent plus dans l'entête.
- ▶ L'accès direct (par exemple en mettant le lien dans la barre d'adresse) soit désactivé.
- ▶ Lors de toute déconnexion, on retourne à la page d'accueil.

5 Page d'accueil

La page d'accueil affiche des informations générales sur le site. Vous êtes relativement libres d'y mettre ce que vous souhaitez (tant qu'il ne s'agit pas d'information qui ne devrait être accessible qu'aux utilisateurs authentifiés !)

Même si c'est la plus simple techniquement, les informations que vous voudriez pouvoir y mettre ne vous seront accessibles qu'à la fin du projet, il faudra donc y revenir. Vous pourrez par exemple y ajouter une boîte avec les *Notes du développeur* : quelles technologies ont été utilisées dans le projet, quelles fonctionnalités/pages en plus ont été ajoutées, quels sont les bugs connus, etc.

6 Les rôles

Dans un système réel, le rôle de chaque utilisateur (répartiteur, traducteur, éditeur) devrait provenir du système d'authentification. Dans le cadre de ce projet, nous n'avons pas accès à de telles informations, le système devra donc *simuler* le changement de rôles, comme si nous étions administrateurs de la plate-forme et pouvions prendre le rôle de tel ou tel.

Le rôle courant, ainsi que dans le cas des traducteurs et éditeurs, l'identité derrière ce rôle, devront être conservées par le site².

La page de *Gestion des rôles* doit donc permettre de changer de rôle pour devenir un répartiteur (rôle par défaut) ou un traducteur/éditeur en particulier.

6.1 Première page, premières requêtes, premiers composants réutilisables

Pour créer cette page, il faut bien entendu récupérer la liste des traducteurs et éditeurs, afin de fournir la liste des choix. Dans les deux cas, il faut mieux ne pas faire la requête à Supabase dans le composant, mais plutôt définir une fonction dans un *service* :

- Dans un (nouveau) fichier `src/services/translatorService.js` créez une fonction `getTranslators` qui fait appel³ à Supabase pour récupérer tous les traducteurs existants.
- Importez cette fonction dans le composant.
- Utilisez-la pour rendre disponible ces données sur la page (pour l'instant cela peut être juste en affichant le tableau retourné).

De même, une fois la liste des personnes récupérée, il faut l'afficher proprement de manière à pouvoir facilement choisir : cela signifie donc la possibilité de trier et filtrer les lignes (et cela devrait vous rappeler des souvenirs des TDs). En particulier, on remarque que le cas des éditeurs et des traducteurs est relativement proche. Pour éviter de recopier du code, il vaut mieux créer un composant réutilisable.

Une fois le choix fait par l'utilisateur (ce qui peut prendre la forme d'un événement à attraper), il doit être répercuté dans l'application, pour que le rôle courant corresponde bien au rôle choisi.

Nous ne vous demanderons plus explicitement d'ajouter des services ou des composants dans la suite de ce document. Nous faisons appel à votre bon sens pour ajouter des fonctions ou/et des services lorsque cela vous semble intéressant, et pour structurer votre code pour éviter les redondances.

2. Il est également envisageable d'utiliser le `localStorage` pour garder l'information lors d'un rechargement.

3. Consultez <https://supabase.com/docs/reference/javascript/select> pour avoir des exemples de requêtes.

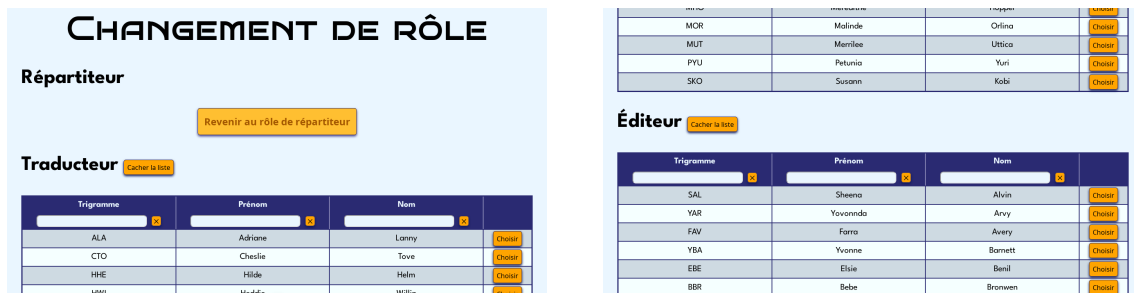


FIGURE 5 – Page de choix d'un rôle (extraits).

6.2 Retour sur le header

Modifier le `header` pour qu'il fasse apparaître en plus de l'identifiant de login, le rôle et éventuellement l'identité liée à ce rôle.



FIGURE 6 – Proposition d'entête (`header`) avec identité et rôle.

7 Les textes

7.1 Navigation par rôles

La *Gestion des textes*, bien que dénotée de la même manière pour tous, ne correspond pas à la même chose pour chaque rôle. En effet, un traducteur ou un éditeur ne doit avoir accès qu'aux textes qui lui sont attribués, et ne peut que marquer un texte comme terminé, alors que le répartiteur, lui voit tous les textes et peut les attribuer. Il faut donc en réalité que cette page soit, à l'instar du site tout entier qui est une *Single Page Application*, une page contenant plusieurs vues.

La méthode pour avoir plusieurs vues n'est pas nouvelle : il faudra y mettre un élément `RouterView`, et le routeur devra se charger d'afficher le bon composant. Pour que le routeur ne confonde pas les vues et les sous-vues, il faut qu'elles soient définies comme telles, avec des *nested routes* (littéralement des *routes imbriquées*, <https://router.vuejs.org/guide/essentials/nested-routes>).

Commencez par mettre en place ces sous-routes, puis vous pourrez vous attaquer au contrôle d'accès. Pour les noms des routes, vous avez le choix mais l'explication ci-dessous utilise :

- `/texts` pour la route de plus haut niveau ;
- `/texts/repartition` pour la sous-route pour le répartiteur ;
- `/texts/traduction` pour la sous-route pour un traducteur ;
- `/texts/edition` pour la sous-route pour un éditeur.

Le comportement attendu par le routage est le suivant :

- L'accès à `/texts` doit rediriger vers la bonne version en fonction du rôle courant, donc par exemple vers `/texts/traduction` s'il s'agit d'un traducteur.

- L'accès à tout autre sous-route doit également rediriger vers la bonne version, donc par exemple en cas d'accès à `/texts/edition` par un traducteur, le routage s'effectue vers `/texts/traduction`.
- Rappelons que si l'utilisateur n'est pas connecté le routage doit de toute manière s'effectuer vers la page d'accueil.

7.2 Pour le répartiteur

7.2.1 Vue globale

La vue globale est un tableau⁴ qui reprend les informations pertinentes sur les textes et leur statut. Une fois de plus, le filtrage et le tri doivent être possible pour améliorer l'ergonomie de la page.

Lorsqu'un texte est choisi, on redirige vers la page de gestion de ce texte, décrite ci-dessous.

GESTION DES TEXTES								
Cote	Titre	Précédent	Reçu le	Limite	Traducteur	Traduit	Éditeur	Terminé
						☐ ☒ ☐ ☐		☐ ☒ ☐ ☐
Co/39-854X	Constitution Act 1902 (New South Wales)		Jeu di 9 avril 2026 à 03:39	Mardi 14 avril 2026 à 05:00	CTO	✗	HRO	✗
Co/38-836F	Constitution of Bosnia and Herzegovina		Jeu di 26 mars 2026 à 16:33	Mercredi 1 avril 2026 à 06:30	HHE	✗	YBA	✗
	Constitution of the		Mercredi 8	Jeu di 23				

FIGURE 7 – Page de gestion globale des textes.

7.2.2 Attribuer un texte

La page de gestion d'un texte utilise son URL pour récupérer l'identifiant du texte. Par exemple `/texts/repartition/text/12` pour le texte d'identifiant 12.

Cette page n'est **pas** routée via une sous-route de `/texts` ! Il faut cependant bien prendre garde à ne pas la laisser accessible par un traducteur ou éditeur, qui sera redirigé vers sa page de gestion des textes.

Sur cette page, il doit être possible de :

- Lire les informations pertinentes sur un texte : métadonnée, dates, délais, attributions existantes, etc.
- Choisir un traducteur et lui fixer une date limite lors de l'attribution. Le système doit avertir le répartiteur s'il fixe une date limite après la date limite globale.
- Choisir un éditeur et l'attribuer au texte.
- Dans les deux cas précédents, la charge de travail actuelle du traducteur ou éditeur doit apparaître pour permettre un choix éclairé au répartiteur.
- Naviguer vers le texte précédent si celui-ci existe.

On remarque que l'on n'a pas de date limite pour l'éditeur : étant la dernière étape, lorsque le traitement du texte est effectué il est directement livré au client. En conséquence, la date limite du texte est aussi la date limite de l'éditeur.

7.3 Pour le traducteur et l'éditeur

Le cas du traducteur et de l'éditeur sont là encore assez proches. Il s'agit tout d'abord d'afficher pour chacun ses textes (c'est à dire ceux qui lui ont été attribués) avec les

4. Tiens, tiens, encore un tableau !

Attribution

Ba/13-488H - 4994 mots - Bail Act 1976

Échéances

Reçu le
Samedi 4 avril 2026 à
08:42

À rendre pour le
Dimanche 12 avril 2026 à
16:51 (reste 10 j, 23 h et
11 min)

Traduction

Traducteur [Montrer les traducteurs](#)

Actuellement sélectionné: Petunia Yuri (PYU) [✕](#)

Délai
07 / 04 / 2026 18 : 00 [⌵](#)

Enregistrer

Attribuer à Petunia Yuri (PYU) pour le Mardi 7 avril 2026 à 18:00, soit un délai de 6 j et 18 min pour 4994 mots (ou 4 j et 3 h ETP à 1200 mots par jour).

Cette personne est déjà occupée à hauteur de 11h ETP. Cela porterait sa file d'attente à 4 j et 14 h.

Valider

Édition

Éditeur [Cacher les éditeurs](#)

Trigramme	Prénom	Nom	À traiter	En attente
SAL	Sheena	Alvin	0	2
YAR	Yovonnda	Arvy	1	2
FAV	Farra	Avery	0	2

FIGURE 8 – Page d'attribution d'un texte (extraits).

Cote	Titre	Disponible	Terminé	Limite	
Ca/75-770Z	Constitution of Berlin of 23 November 1995, as amended as of 22 March 2016.	✓	✗	Lundi 30 mars 2026 à 06:36	Marquer comme terminé
Ca/69-749Y	Constitution of India (Original Calligraphed and Illuminated Version)	✓	✗	Mercredi 29 avril 2026 à 08:14	Marquer comme terminé
Se/74-719F	Socialist Constitution of the Democratic People's Republic of Korea (1972)	✗	✗	Vendredi 10 avril 2026 à 00:17	Marquer comme terminé

Cote	Titre	Limite	Précédent	Terminé	
Ca/75-770Z	Constitution of Berlin of 23 November 1995, as amended as of 22 March 2016.	Samedi 28 mars 2026 à 14:07	Ca/53-824P	✓	Marquer comme traduit
Ca/38-764K	Constitution of North Rhine-Westphalia	Dimanche 5 avril 2026 à 17:37		✓	Marquer comme traduit
Ca/59-761Z	Constitution of the Kingdom of Prussia (1848, revised 1950)	Dimanche 19 avril 2026 à 17:46		✓	Marquer comme traduit
Ca/28-758C	Constitution of Ghana (1992)	Lundi 6 avril 2026 à 06:06		✗	Marquer comme traduit
Pr/50-662Q	Provisional Constitution of the Philippines	Dimanche 12 avril 2026 à 14:14		✓	Marquer

FIGURE 9 – Page de gestion des textes pour un éditeur (gauche) et un traducteur (droite).

informations pertinentes (qui peuvent varier selon le rôle). Il faut ensuite que l'on puisse valider la fin d'un texte (traduction ou édition, selon le cas). Attention, il n'est pas possible pour un éditeur de traiter un texte qui n'est pas encore traduit !

8 Statistiques

La dernière partie du système de gestion des textes doit permettre d'afficher des statistiques de productivité individuelles et globales : productivité, textes rendus en retard, textes en attente... Une fois encore, les rôles sont importants :

- le répartiteur peut voir les statistiques globales ainsi que les statistiques individuelles de tout le monde,
- les autres ne peuvent voir que leurs propres statistiques individuelles.

Ces règles sont gérées par le routeur. De plus, pour les statistiques individuelles, l'identifiant de l'objet des statistiques (un trigramme, donc) fait partie de l'URL de la page. Le répartiteur doit pouvoir choisir sur un tableau le traducteur ou éditeur dont il souhaite voir les statistiques.

Un paramètre important qui doit figurer sur cette page est la possibilité de choisir l'intervalle de temps qui servira à faire les statistiques. Le reste des statistiques est laissé à votre guise. Pour cette page, vous serez sans doute amené à définir de nouvelles vues ou fonctions directement dans la base de donnée.

Les requêtes pour calculer ces statistiques peuvent être nombreuses et/ou coûteuses. Il est sans doute préférable de demander à l'utilisateur de valider un changement de l'intervalle ; on perd en réactivité mais on évite de surcharger inutilement la base de données.

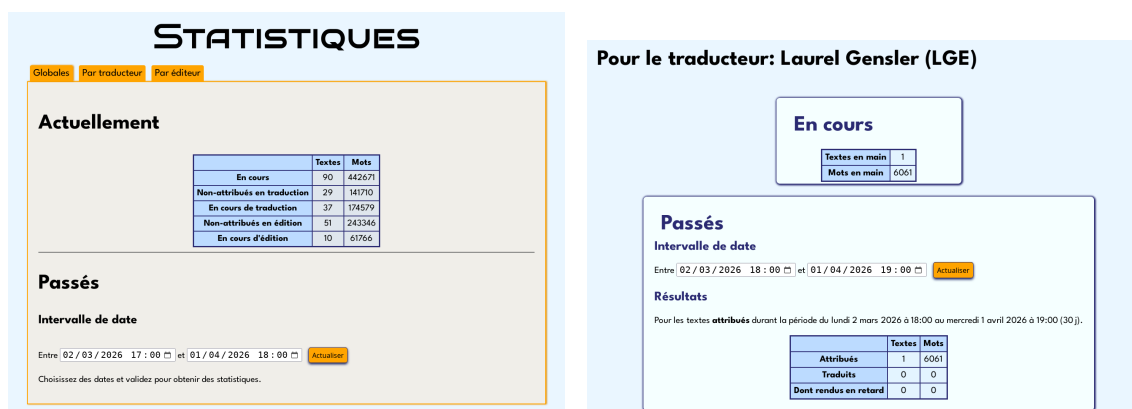


FIGURE 10 – Page de statistiques globales (gauche) et pour un traducteur (droite).

9 Attendus et consignes

9.1 Fonctionnalités de base

Les pages qui ont été décrites, ainsi que leur bon interfaçage avec la base de donnée constituent le minimum pour obtenir un site fonctionnel répondant aux attentes du client ⁵.

Le site doit être fonctionnel dès son déploiement, et ne pas produire d'erreurs ou d'avertissements lors de l'exécution.

Nous serons également attentifs à la qualité du code et à sa structure : utilisez-donc le *framework* et ses modules (vue-router, Pinia) à bon escient, et décomposez en composants lorsque cela est pertinent. Le code doit être clair, indenté, et commenté.

Assurez vous donc, avant la remise (et avant de passer au point 9.3!), de répondre à ces attentes.

9.2 Style et ergonomie

Un CSS minimal qui permet une lecture fluide de vos pages est exigé. Pensez également aux aspects UX ⁶ : l'utilisateur est-il averti lorsqu'une requête n'aboutit pas ? etc. Un CSS qui en fait un « beau » site est mieux (et relativement facile à obtenir avec un framework CSS ⁷).

9.3 Fonctionnalités avancées

Vous pouvez bien sûr partir de cette base pour enrichir le projet. Voici quelques pistes, mais si vous avez d'autres idées, n'hésitez pas à en faire part à votre enseignant de labo, avec qui vous pourrez échanger sur la pertinence et la faisabilité de vos propositions. Tous ces points et d'autres qui ne font pas partie de ce projets ne font pas partie des fonctionnalités requises.

Ces améliorations ne sont pas demandées dans l'énoncé et ne sont donc pas exigées.

Rôle client Ajouter un quatrième rôle : le client. C'est lui qui introduit les demandes de traduction et fixe donc les délais globaux. Il n'a pas accès aux autres pages, et à l'inverse sa page de demande de traduction n'est pas accessible aux traducteurs, éditeurs, et au répartiteur.

5. Ici, votre enseignant de labo.

6. *User experience*.

7. *tailwindcss*, *headlessUI*, *heroicons*...

Thème par rôle Avoir un thème CSS par rôle du site. Ainsi, le répartiteur serait (par exemple) sur un site plutôt bleu, alors que le traducteur est rouge et l'éditeur vert.

Modifications des attributions Selon ce qui a été présenté, une fois les attributions créées, il n'est plus possible de revenir dessus. Une amélioration serait de pouvoir le faire, mais avec des limites et délais *raisonnables* : on ne veut pas retirer une attribution à quelqu'un qui aurait déjà commencé à travailler sur le texte !

CSS sémantique Certes des classes permettent déjà de donner une représentation visuelle des valeurs, mais cette idée va plus loin : utiliser les valeurs des délais restants pour colorier de manière continue des valeurs en fonction de l'urgence de la situation (vert : il y a le temps ; rouge ça urge ; violet : on est en retard !).

9.4 IA et autres sources

Il ne vous est pas interdit d'utiliser l'IA, comme il ne vous est pas interdit d'utiliser des solutions postées sur des forums d'entraide (type StackOverflow) ou même la documentation officielle des technologies que vous utilisez. Néanmoins :

- ▶ Tout code qui n'a pas subi de modification substantielle doit être sourcé.
- ▶ Vous devrez être en mesure d'expliquer votre code lors de la défense, et de pouvoir le refaire ou faire du code similaire.

Rappel

La note se base sur la défense en utilisant le projet comme support ; il n'y a pas de points attribués spécifiquement au résultat du projet.

9.5 Remise

La remise s'effectue sur le dépôt `git` qui a été créé pour vous à cet effet. Il est attendu des commits réguliers et *atomiques* : un commit ne devrait mentionner qu'une seule chose, et cela devrait d'ailleurs se voir dans les messages de commit. Le dépôt doit contenir tout ce qui permettra à l'enseignant de tester votre projet : il devrait suffire de faire `npm install`; `npm run dev` depuis un clone frais pour lancer le site.

Si vous ajoutez des vues ou fonctions à votre base de donnée, les scripts permettant de les créer doivent se trouver dans un dossier **ressource** à la racine de votre projet Vue.

La date limite de la remise ainsi que les modalités de défense vous seront communiquées via poÉSI.